

# Giới thiệu về policy gradient

DEEP REINFORCEMENT LEARNING BẰNG PYTHON



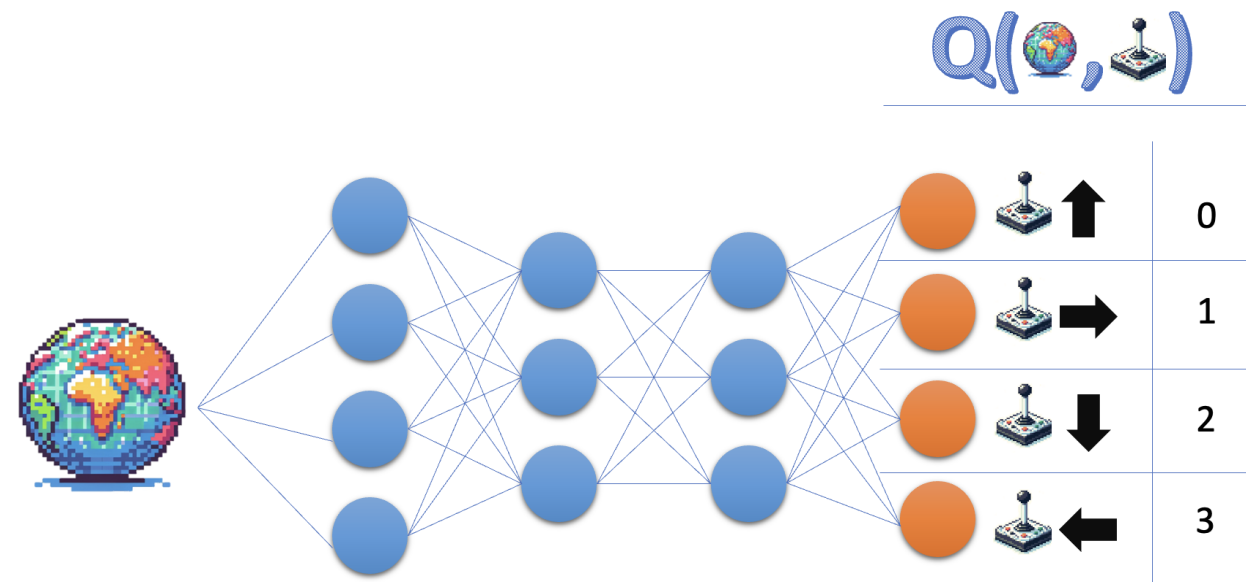
**Timothée Carayol**

Principal Machine Learning Engineer,  
Komment

# Giới thiệu về phương pháp Policy trong DRL

## Q-learning:

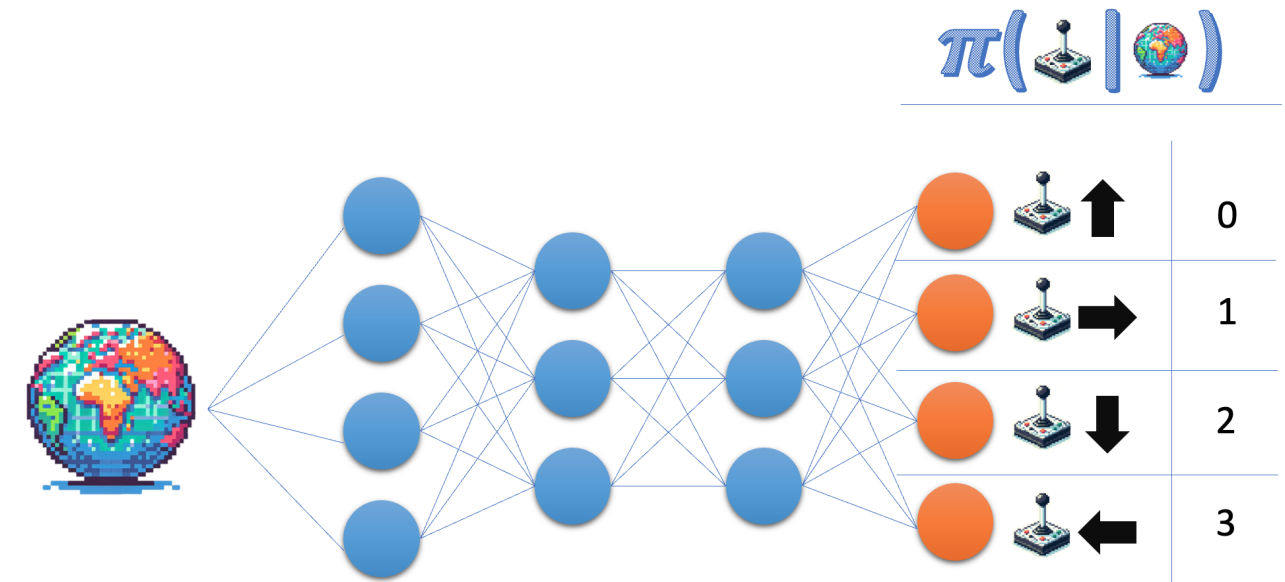
- Học hàm giá trị hành động Q



- Policy: chọn hành động có giá trị cao nhất

## Policy learning:

- Học trực tiếp policy



# Học policy

- Có thể ngẫu nhiên
  - Xử lý không gian liên tục
  - Tối ưu trực tiếp theo mục tiêu
  - Phương sai cao
  - Kém hiệu quả mẫu hơn
- 
- Trong Deep-Q learning: policy là xác định

$$\pi_{\theta}(a_t | s_t):$$

- Phân phối xác suất cho  $a_t$  tại trạng thái  $s_t$ , với:
  - $a_t, s_t$ : hành động và trạng thái tại bước  $t$
  - $\theta$ : tham số policy (trọng số mạng)

# Policy network (hành động rời rạc)

```
class PolicyNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, state):
        x = torch.relu(self.fc1(torch.tensor(state)))
        x = torch.relu(self.fc2(x))
        action_probs = torch.softmax(self.fc3(x), dim=-1)
        return action_probs
```

```
action_probs = policy_network(state)
print('Action probabilities:', action_probs)
```

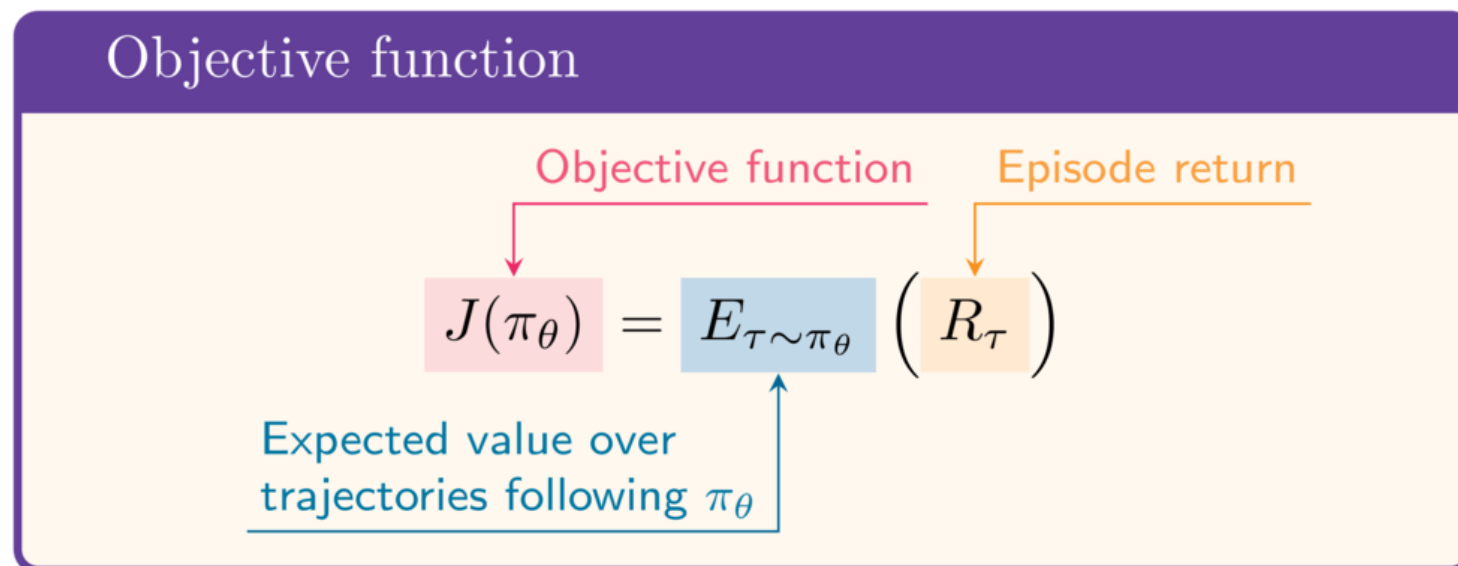
```
Action probabilities: tensor([0.21, 0.02, 0.74, 0.03])
```

| Action                                                                                                                                                                                                                                                         | index | $\pi(\text{ joystick }   \text{ globe })$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-------------------------------------------|
|       | 0     | 0.21                                      |
|       | 1     | 0.02                                      |
|       | 2     | 0.74                                      |
|    | 3     | 0.03                                      |

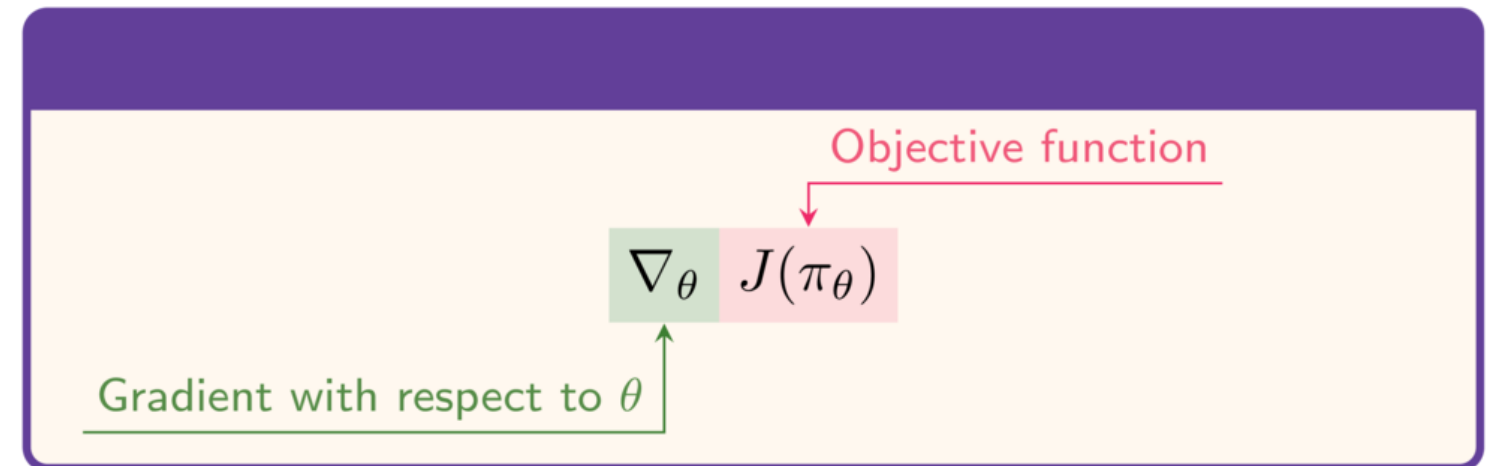
```
action_dist = (
    torch.distributions.Categorical(action_probs))
action = action_dist.sample()
```

# Hàm mục tiêu

- Policy phải tối đa hóa lợi nhuận kỳ vọng
  - Giả sử tác tử theo  $\pi_\theta$
  - Bằng cách tối ưu tham số policy  $\theta$
- Hàm mục tiêu:

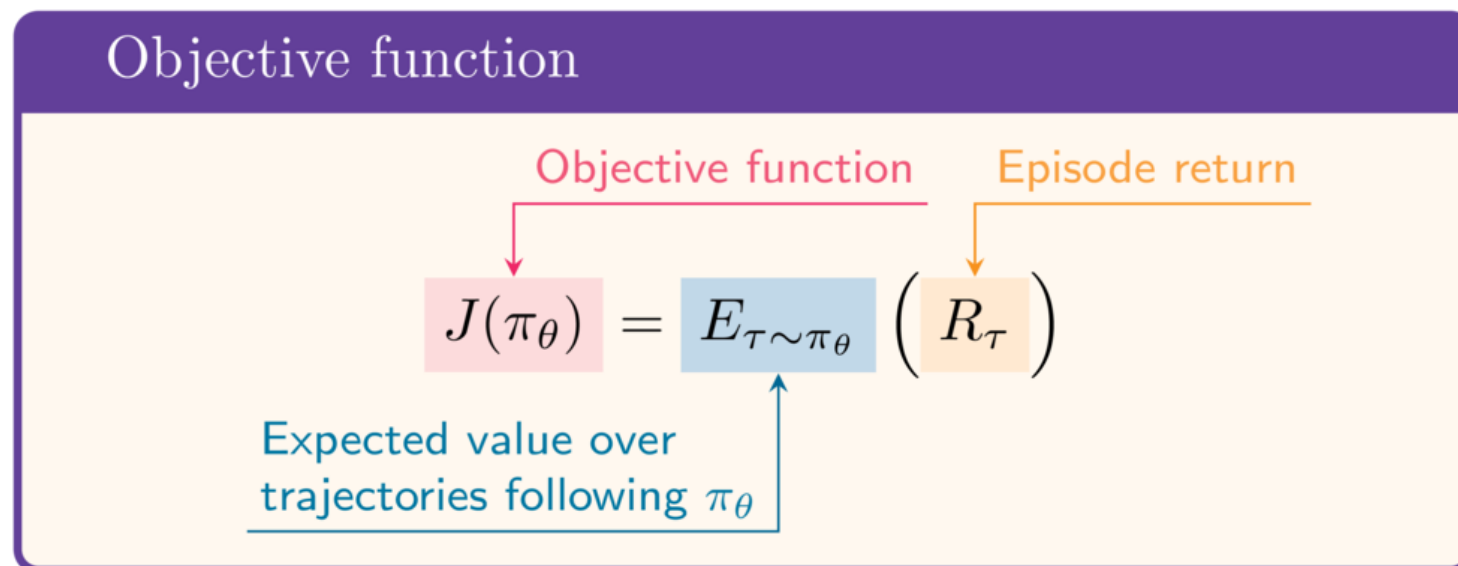


- Để tối đa hóa  $J$ : cần gradient theo  $\theta$ :

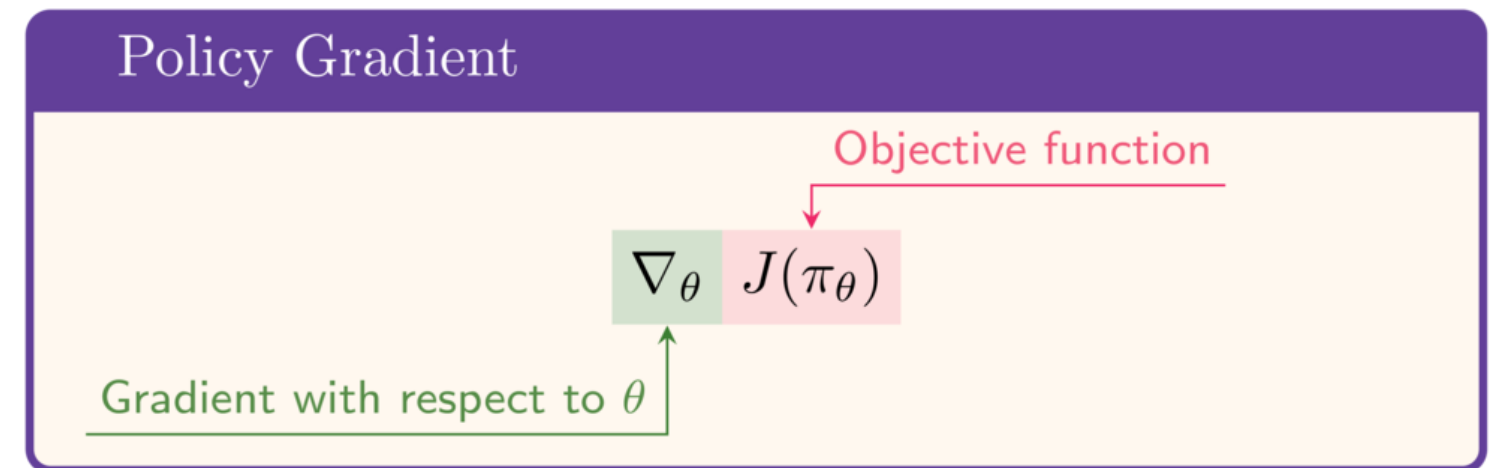


# Hàm mục tiêu

- Policy phải tối đa hóa lợi nhuận kỳ vọng
  - Giả sử tác tử theo  $\pi_\theta$
  - Bằng cách tối ưu tham số policy  $\theta$
- Hàm mục tiêu:



- Để tối đa hóa  $J$ : cần gradient theo  $\theta$ :



# Định lý policy gradient

- Cho biểu thức tính được cho  $\nabla_{\theta} J(\pi_{\theta})$
- Kỳ vọng trên các quỹ đạo theo  $\pi_{\theta}$ 
  - Thu thập quỹ đạo và quan sát tổng thưởng

## Policy Gradient Theorem

The diagram shows the equation  $\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( \right)$  inside a light yellow box with a purple border. A green arrow labeled "Policy gradient" points to the  $\nabla_{\theta} J(\pi_{\theta})$  term. A blue arrow labeled "Expected value over trajectories following  $\pi_{\theta}$ " points to the  $E_{\tau \sim \pi_{\theta}}$  term. The right side of the equation is an empty set of parentheses.

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( \right)$$

# Định lý policy gradient

- Cho biểu thức tính được cho  $\nabla_{\theta} J(\pi_{\theta})$
- Kỳ vọng trên các quỹ đạo theo  $\pi_{\theta}$ 
  - Thu thập quỹ đạo và quan sát tổng thưởng
- Với mỗi quỹ đạo: xét tổng thưởng  $R_{\tau}$

Policy Gradient Theorem

The diagram shows the equation  $\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( R_{\tau} \right)$  inside a light yellow box with a purple border. A green arrow labeled 'Policy gradient' points to the  $\nabla_{\theta} J(\pi_{\theta})$  term. An orange arrow labeled 'Episode return' points to the  $R_{\tau}$  term. A blue arrow labeled 'Expected value over trajectories following  $\pi_{\theta}$ ' points to the expectation operator  $E_{\tau \sim \pi_{\theta}}$ .

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( R_{\tau} \right)$$

Expected value over trajectories following  $\pi_{\theta}$

# Định lý policy gradient

- Cho biểu thức tính được cho  $\nabla_{\theta} J(\pi_{\theta})$
- Kỳ vọng trên các quỹ đạo theo  $\pi_{\theta}$ 
  - Thu thập quỹ đạo và quan sát tổng thưởng
- Với mỗi quỹ đạo: xét tổng thưởng  $R_{\tau}$
- Nhân với tổng gradient của log xác suất các hành động đã chọn
- Trực giác: chỉnh  $\theta$  để tăng xác suất các hành động trong tập “tốt”

Policy Gradient Theorem

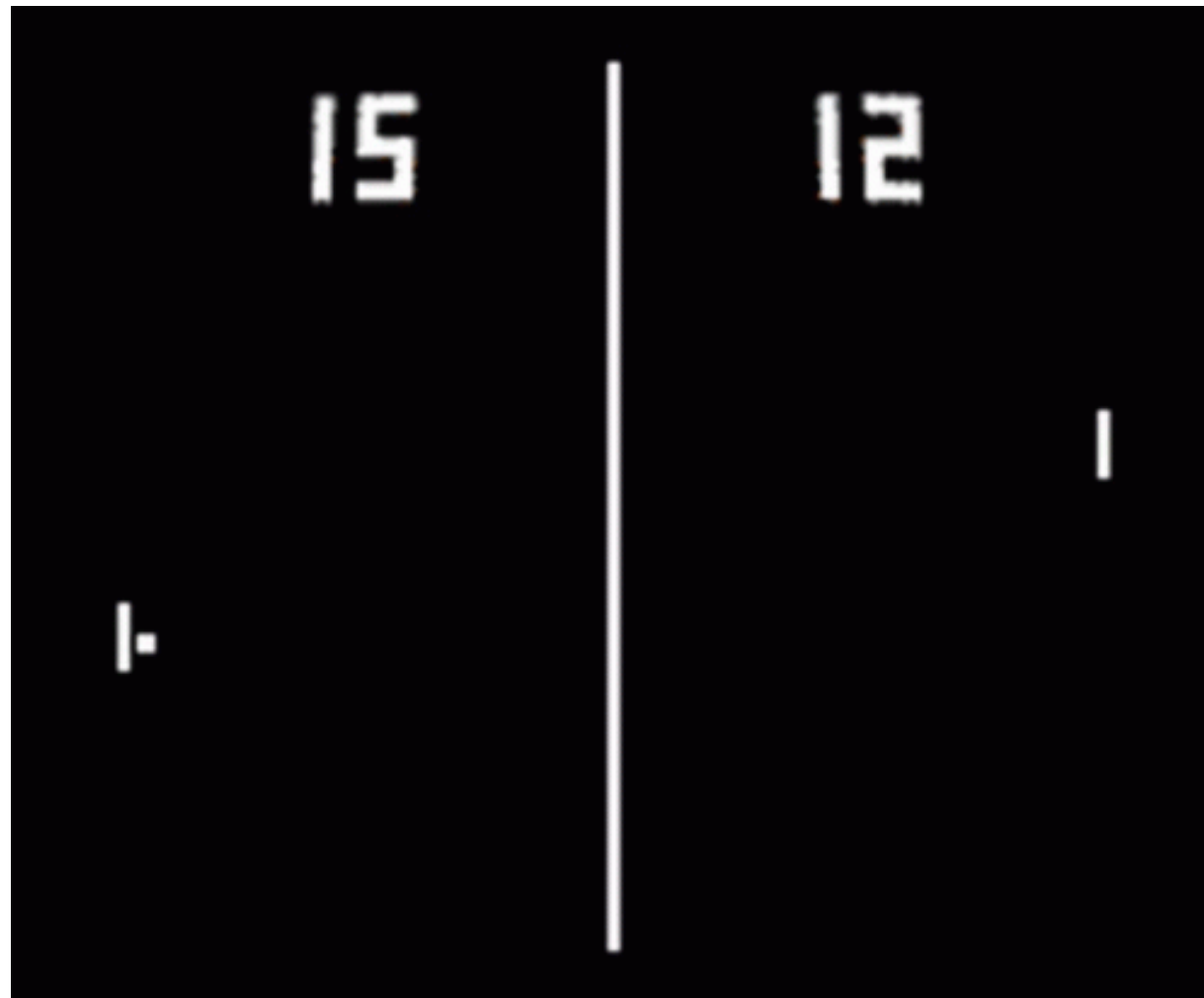
The diagram illustrates the Policy Gradient Theorem equation:  $\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( R_{\tau} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)$ . Annotations include: a green arrow pointing to  $\nabla_{\theta} J(\pi_{\theta})$  labeled 'Policy gradient'; a blue arrow pointing to  $E_{\tau \sim \pi_{\theta}}$  labeled 'Expected value over trajectories following  $\pi_{\theta}$ '; an orange arrow pointing to  $R_{\tau}$  labeled 'Episode return'; and a red arrow pointing to  $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$  labeled 'Gradient of action log probability'.

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( R_{\tau} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)$$

Expected value over trajectories following  $\pi_{\theta}$

Episode return

Gradient of action log probability



# Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

# Gradient theo chính sách và REINFORCE

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

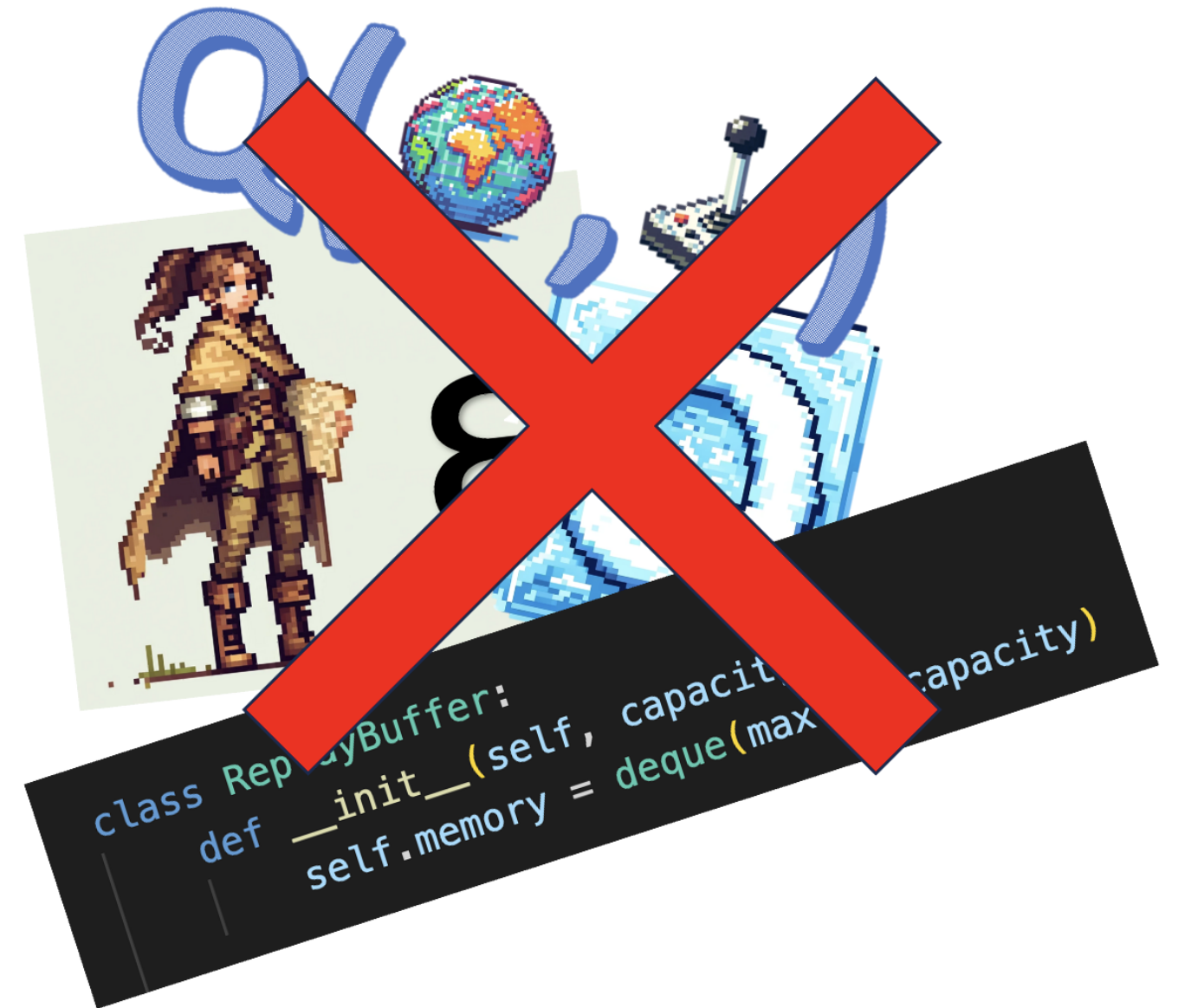


**Timothée Carayol**

Principal Machine Learning Engineer,  
Komment

# Khác biệt so với DQN

- REINFORCE: Monte Carlo, không phải Temporal Difference
  - Cập nhật cuối mỗi tập, không phải mỗi bước
  - Hoặc cập nhật sau nhiều tập
- Không có hàm giá trị
- Không có mạng mục tiêu
- Không epsilon-greedy
- Không replay kinh nghiệm



# Cấu trúc vòng lặp huấn luyện REINFORCE

```
for episode in range(num_episodes):  
    # 1. Khởi tạo tập  
    while not done:  
        # 2. Chọn hành động  
        # 3. Thực thi và lấy trạng thái, phần thưởng kế  
        # 4. Cộng phần thưởng (chiết khấu) vào tổng lợi tức  
        # 5. Cập nhật trạng thái  
    # 6. Tính loss  
    # 7. Cập nhật mạng chính sách bằng hạ dốc
```

# Chọn hành động

```
from torch.distributions import Categorical

def select_action(policy_network, state):
    action_probs = policy_network(state)
    action_dist = Categorical(action_probs)
    action = action_dist.sample()
    log_prob = action_dist.log_prob(action)
    return action.item(), log_prob.reshape(1)

action, log_prob = select_action(
    policy_network, state)
```

- Lấy xác suất từ mạng
- Lấy mẫu một hành động
- Trả về hành động và log-prob tương ứng

Chỉ số hành động được lấy mẫu: 1  
Log-prob của hành động đã lấy mẫu: -1.38

# Tính loss

Nhắc lại định lý gradient theo chính sách:

Policy Gradient Theorem

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left( R_{\tau} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)$$

Expected value over trajectories following  $\pi_{\theta}$

Policy gradient

Episode return

Gradient of action log probability

REINFORCE Loss function

$$\mathcal{L}(\theta) = -R_{\tau} \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t)$$

Loss

Episode return

Sum of action log probabilities

Trong Python:

- $R_{\tau}$  là `episode_return`
- Vector  $\log \pi_{\theta}(a_t | s_t)$  là `episode_log_probs`

```
loss = -episode_return * episode_log_probs.sum()
```

# Vòng lặp huấn luyện REINFORCE

```
for episode in range(50):
    state, info = env.reset(); done = False; step = 0;
    episode_log_probs = torch.tensor([])
    R = 0
    while not done:
        step += 1
        action, log_prob = select_action(policy_network, state)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        R += (gamma ** step) * reward
        episode_log_probs = torch.cat((episode_log_probs, log_prob))
        state = next_state
    loss = - R * episode_log_probs.sum()
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

# Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

# Advantage Actor Critic

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

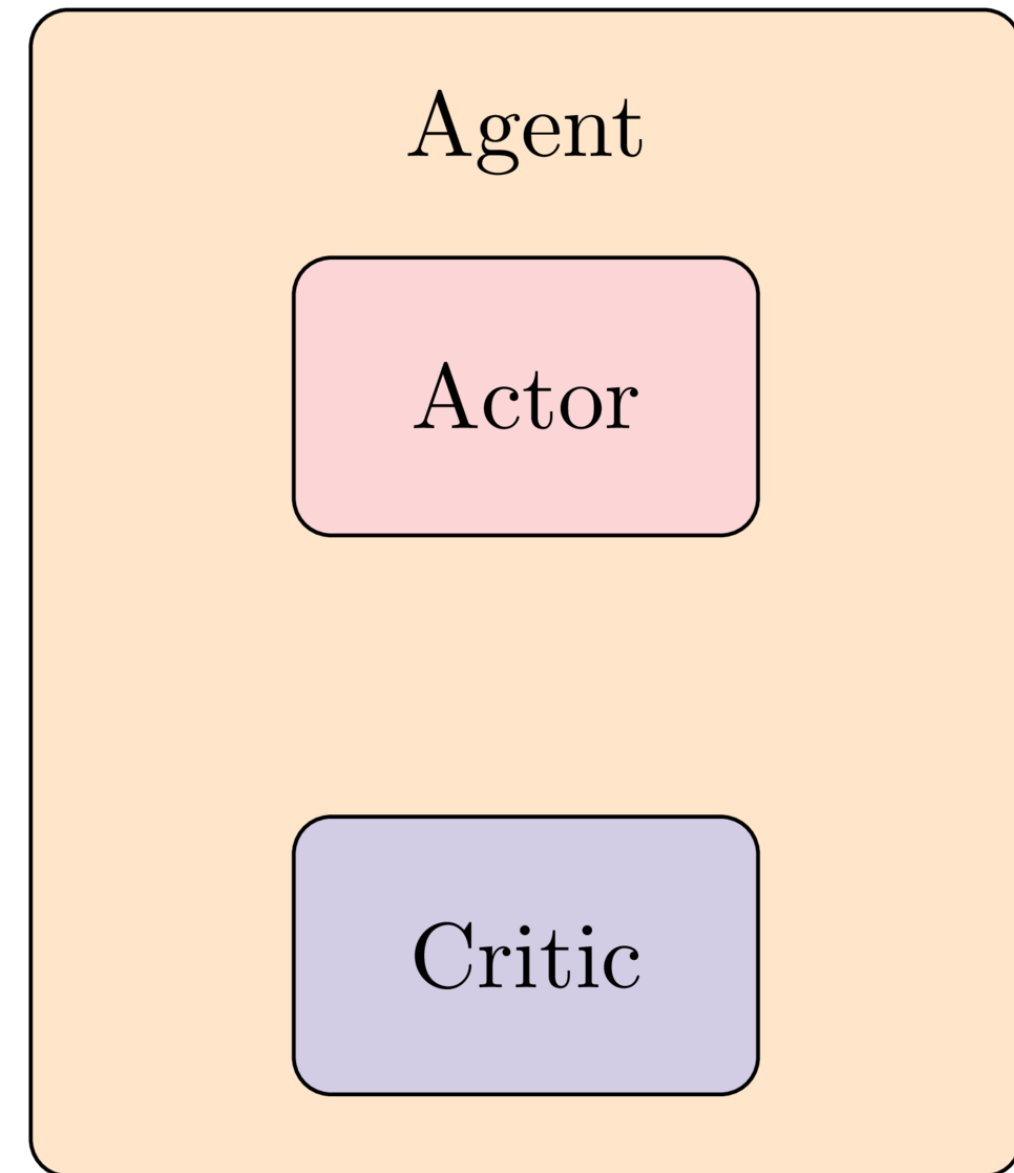


**Timothée Carayol**

Principal Machine Learning Engineer,  
Komment

# Vì sao Actor–Critic?

- Hạn chế của REINFORCE:
  - Phương sai cao
  - Hiệu quả mẫu kém
- Phương pháp Actor–Critic thêm mạng critic, cho phép học Chênh lệch Thời gian (TD)



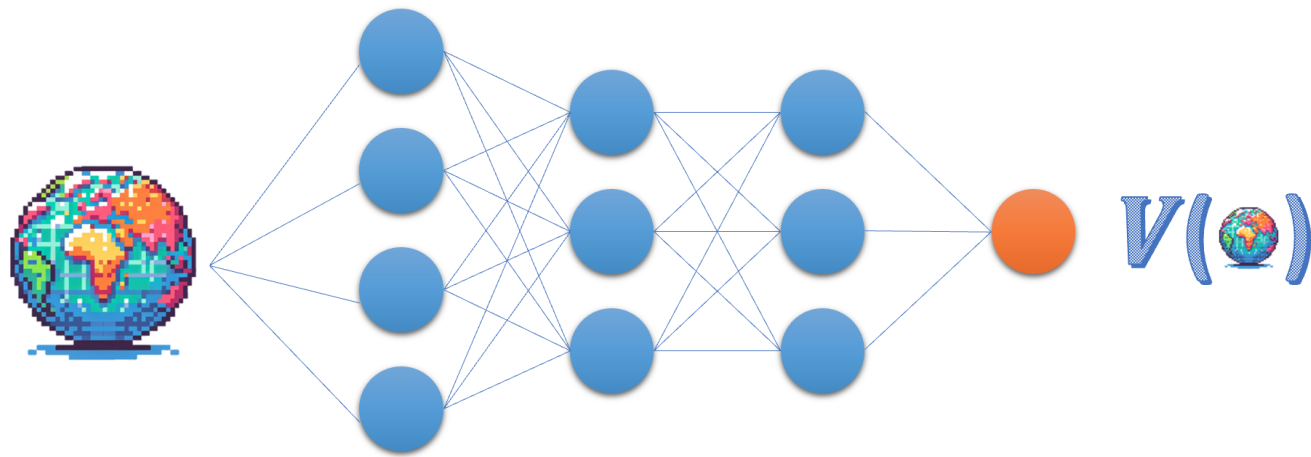
# Trực giác về phương pháp Actor–Critic



- Mạng Actor:
  - Ra quyết định
  - Không tự đánh giá được
- Mạng Critic:
  - Phản hồi cho actor ở mỗi bước

# Mạng Critic

- Critic xấp xỉ hàm giá trị trạng thái

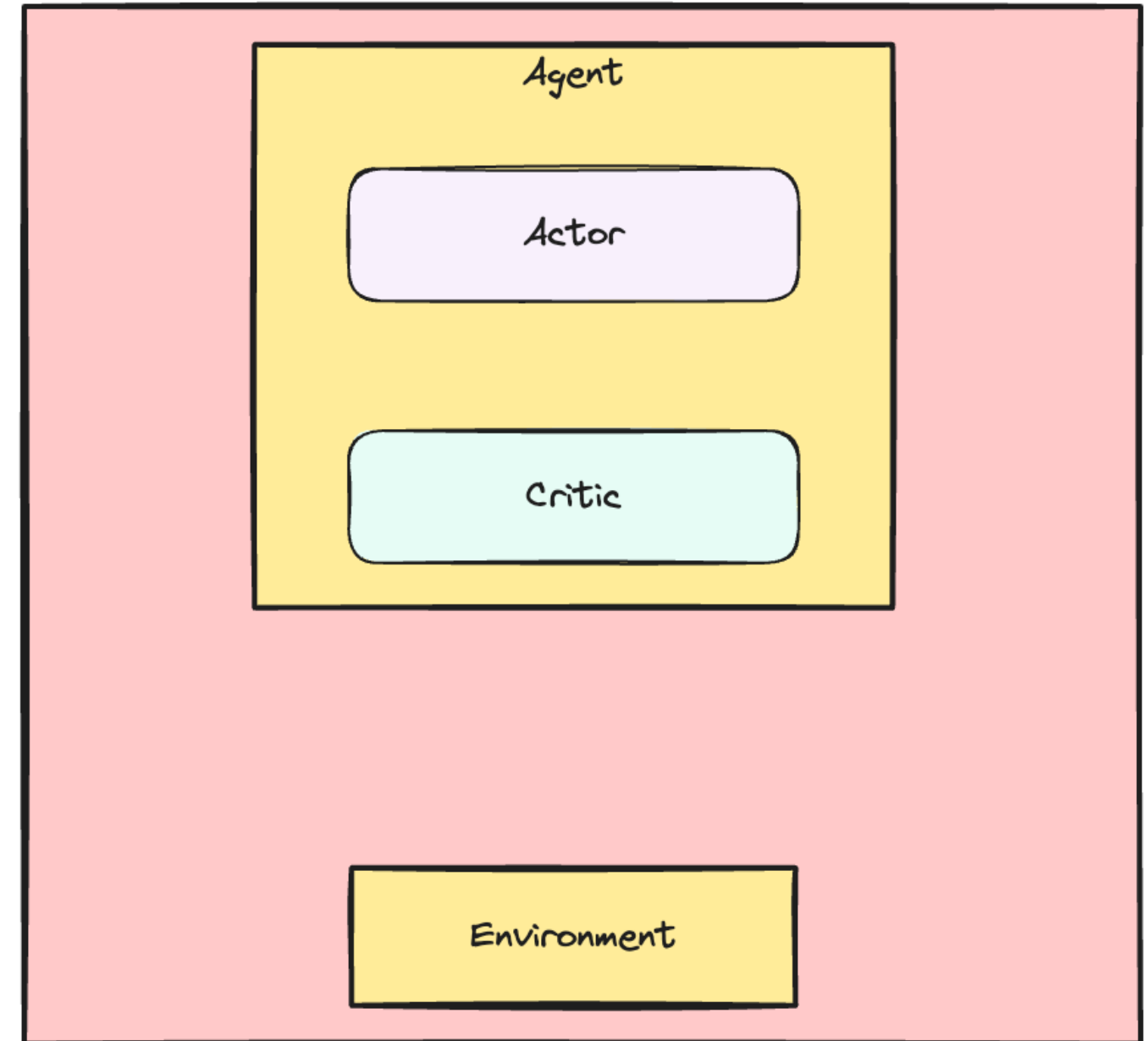


- Đánh giá hành động  $a_t$  dựa trên lợi thế hoặc sai số TD

```
class Critic(nn.Module):  
    def __init__(self, state_size):  
        super(Critic, self).__init__()  
        self.fc1 = nn.Linear(state_size, 64)  
        self.fc2 = nn.Linear(64, 1)  
  
    def forward(self, state):  
        x = torch.relu(self.fc1(torch.tensor(state)))  
        value = self.fc2(x)  
        return value  
  
critic_network = Critic(8)
```

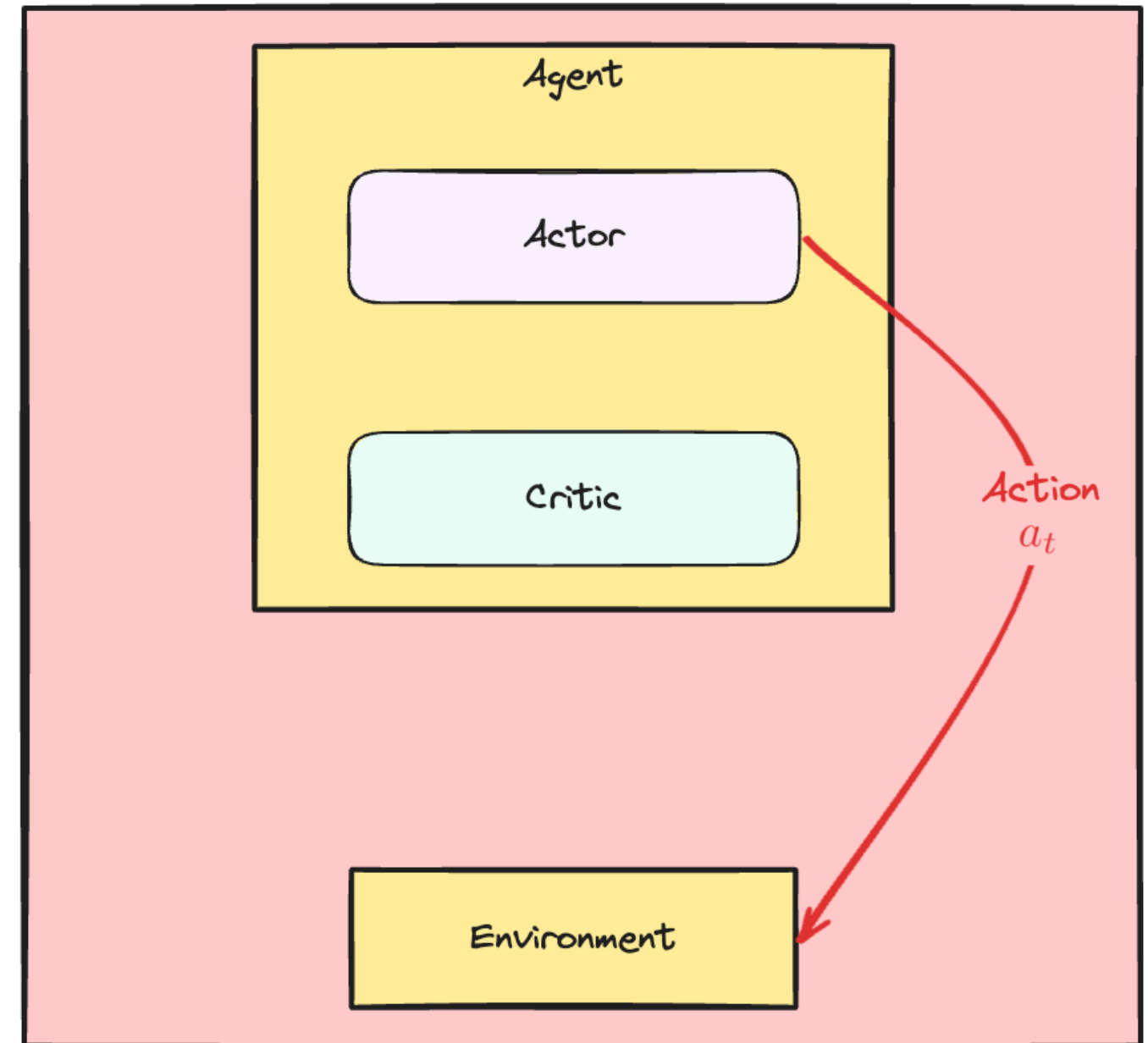
# Động lực Actor–Critic

- Mỗi bước:
  - Actor chọn hành động (giống mạng chính sách trong REINFORCE)



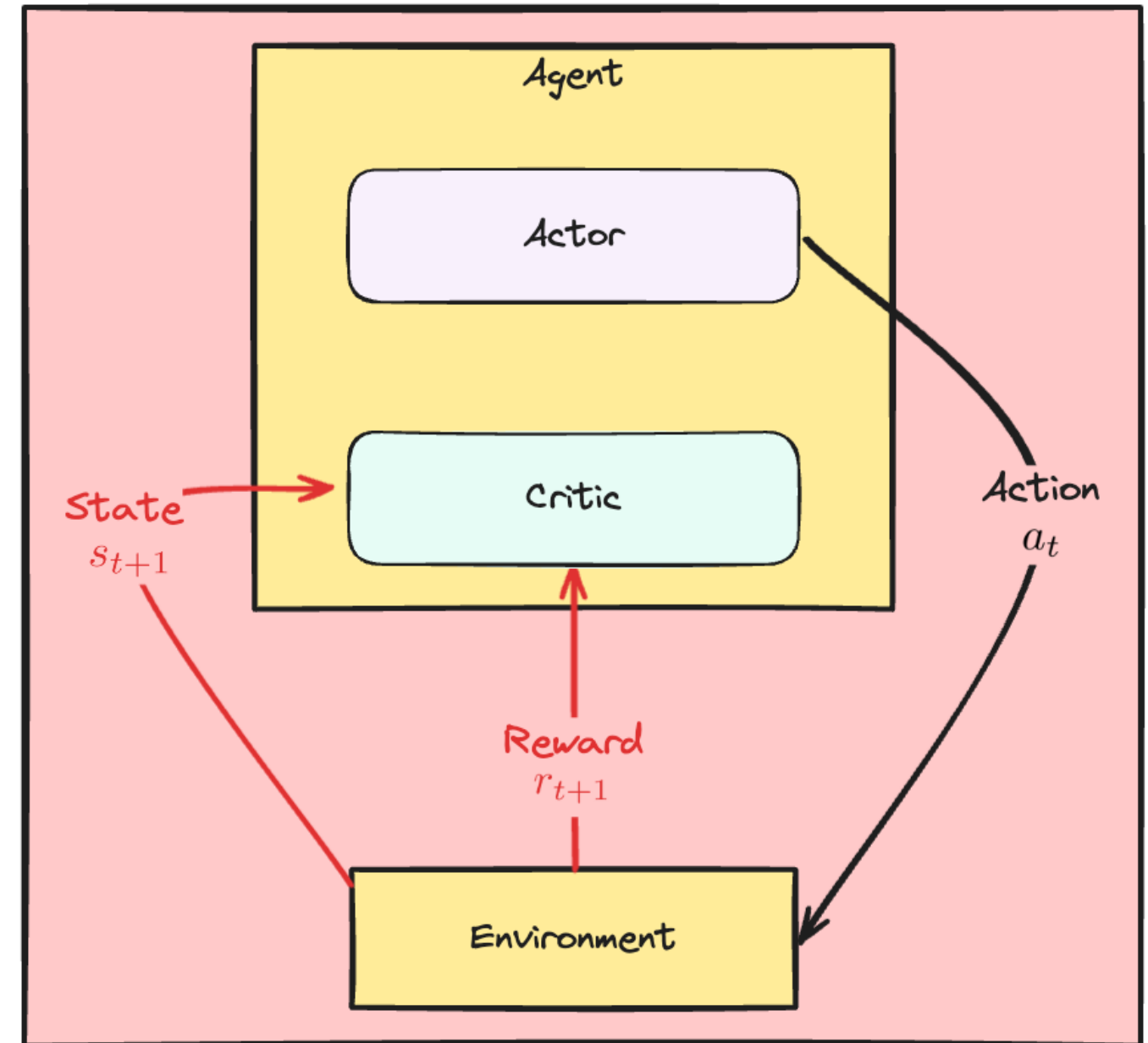
# Động lực Actor–Critic

- Mỗi bước:
  - Actor chọn hành động (giống mạng chính sách trong REINFORCE)
  - Critic quan sát phần thưởng và trạng thái



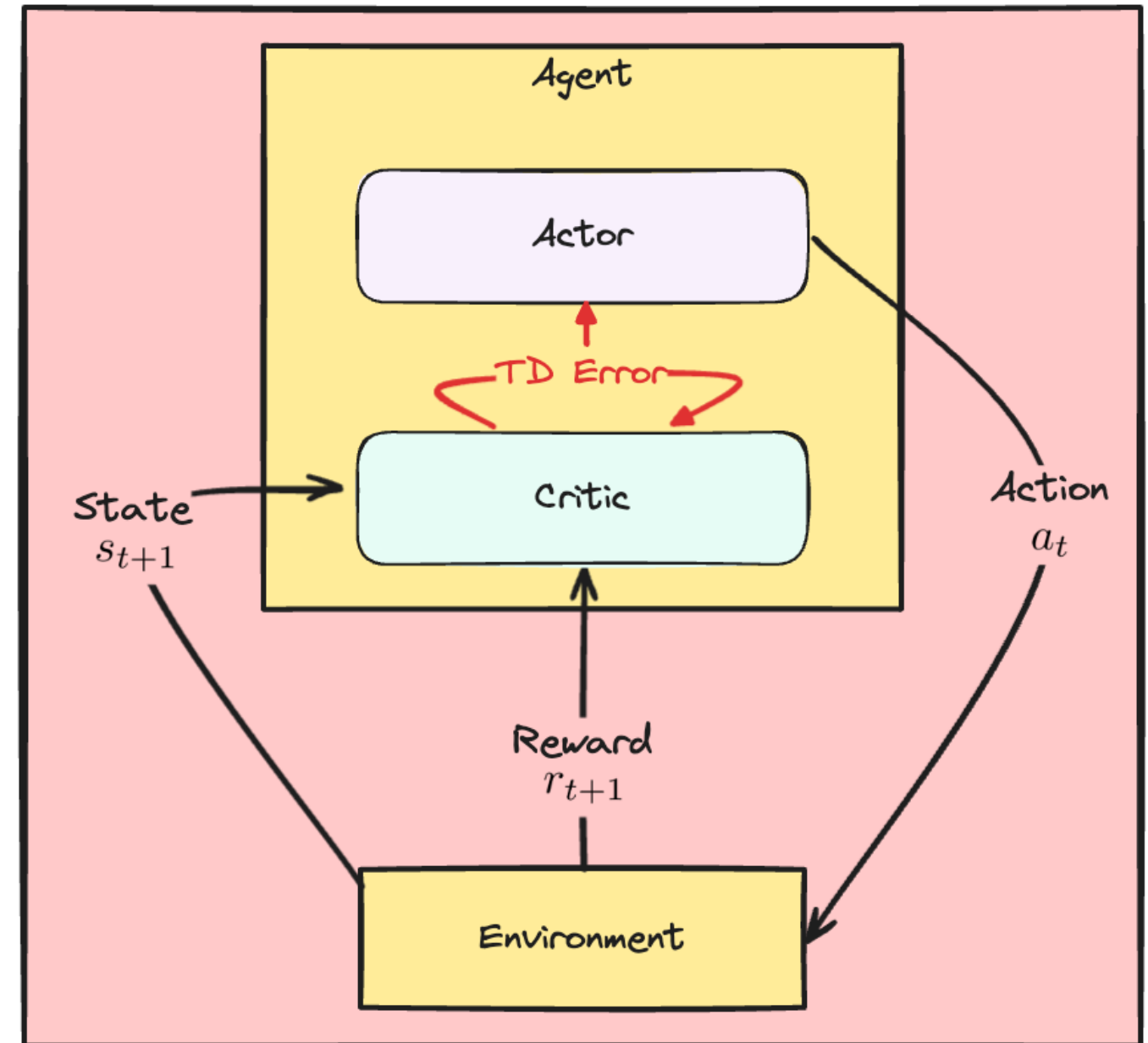
# Động lực Actor–Critic

- Mỗi bước:
  - Actor chọn hành động (giống mạng chính sách trong REINFORCE)
  - Critic quan sát phần thưởng và trạng thái
  - Critic tính sai số TD
  - Actor và Critic dùng sai số TD để cập nhật trọng số



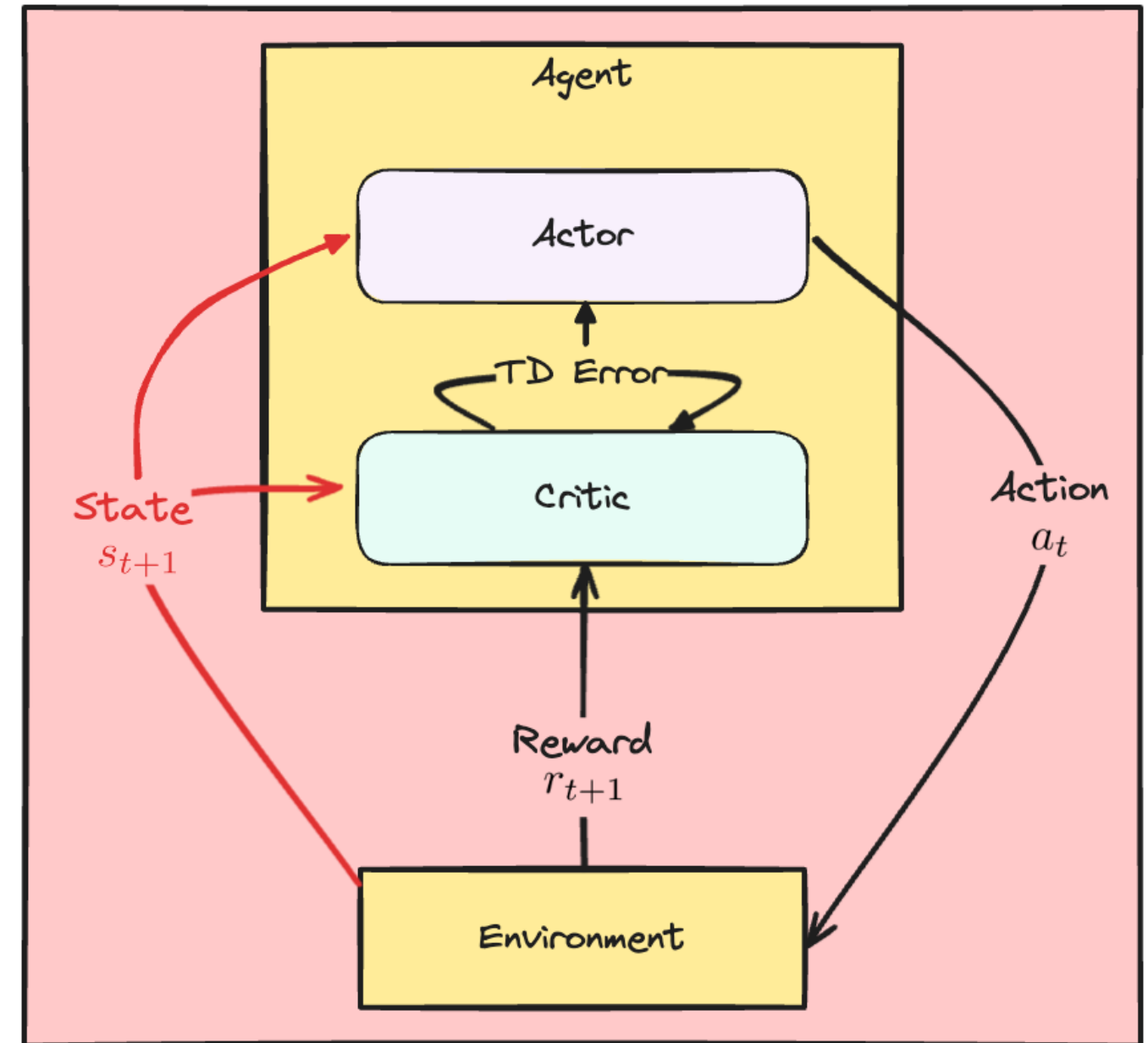
# Động lực Actor–Critic

- Mỗi bước:
  - Actor chọn hành động (giống mạng chính sách trong REINFORCE)
  - Critic quan sát phần thưởng và trạng thái
  - Critic tính sai số TD
  - Actor và Critic dùng sai số TD để cập nhật trọng số
  - Actor đã cập nhật quan sát trạng thái mới



# Động lực Actor–Critic

- Mỗi bước:
  - Actor chọn hành động (giống mạng chính sách trong REINFORCE)
  - Critic quan sát phần thưởng và trạng thái
  - Critic tính sai số TD
  - Actor và Critic dùng sai số TD để cập nhật trọng số
  - Actor đã cập nhật quan sát trạng thái mới



# Hàm mất mát A2C

## Critic

### Critic loss function

Use the Squared TD Error for the Critic:

$$L_c(\theta_c) = \left( \overbrace{(r_t + \gamma V_{\theta_c}(s_{t+1})) - V_{\theta_c}(s_t)}^{\text{TD error / Advantage}} \right)^2$$

$\theta_c$  is labeled "Parameters of the critic".

- Mất mát critic: sai số TD bình phương

## Actor

### Actor loss function

It can be shown that we can use, at every step  $t$ , this loss function for the Actor:

$$L(\theta) = - \log \pi_{\theta}(a_t | s_t) \left( \overbrace{(r_t + \gamma V(s_{t+1})) - V(s_t)}^{\text{TD error / Advantage}} \right)$$

$\theta$  is labeled "Parameters of the actor".  
 $\log \pi_{\theta}(a_t | s_t)$  is labeled "Action log probability".

- Sai số TD phản ánh đánh giá của critic
- Tăng xác suất các hành động có sai số TD dương

# Tính các hàm mất mát

```
def calculate_losses(critic_network, action_log_prob,
                    reward, state, next_state, done):
    # Critic provides the state value estimates
    value = critic_network(state)
    next_value = critic_network(next_state)
    td_target = (reward + gamma *
                 next_value * (1-done))
    td_error = td_target - value

    # Apply formulas for actor and critic losses
    actor_loss = -action_log_prob * td_error.detach()
    critic_loss = td_error ** 2

    return actor_loss, critic_loss
```

- Tính sai số TD
- Tính mất mát actor
  - Dùng `.detach()` để chặn lan truyền gradient sang trọng số critic
- Tính mất mát critic

# Vòng lặp huấn luyện Actor–Critic

```
for episode in range(10):
    state, info = env.reset()
    done = False
    while not done:
        # Select action
        action, action_log_prob = select_action(actor, state)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        # Calculate losses
        actor_loss, critic_loss = calculate_losses(critic, action_log_prob, reward, state, next_state, done)
        # Update actor
        actor_optimizer.zero_grad(); actor_loss.backward(); actor_optimizer.step()
        # Update critic
        critic_optimizer.zero_grad(); critic_loss.backward(); critic_optimizer.step()
    state = next_state
```

# Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON